```
#include <iostream>
#include <string>
#include <algorithm>

using namespace std;

struct Node {
   char data;
   Node* next;
};

class Stack {
private:
   Node* top = nullptr;
public:
   bool isEmpty() {
      return top == nullptr;
   }

   void push(char val) {
      Node* newNode = new Node();
      newNode->data = val;
      newNode->next = top;
      top = newNode;
   }

   char pop() {
      if (isEmpty()) return '\0';

      Node* temp = top;
      char val = temp->data;
      top = top->next;
      delete temp;
      return val;
   }

   char peek() {
      if (isEmpty()) return '\0';
      return top->data;
   }
};

int getPrecedence(char op) {
   if (op == '+' || op == '-') return 1;
   if (op == '*' || op == '/') return 2;
   if (op == '^') return 3;
   return 0;
}
```

```
bool isOperand(char ch) {
   return (ch >= 'a' && ch <= 'z') ||
         (ch >= 'A' && ch <= 'Z') ||
         (ch >= '0' && ch <= '9');
}

string infixToPostfix(string expression) {
   Stack s;
   string postfix = "";

   for (int i = 0; i < expression.length(); i++) {
      char ch = expression[i];

      if (isOperand(ch)) {
         postfix += ch;
      }
      else if (ch == '(') {
         s.push(ch);
      }
      else if (ch == ')') {
         while (!s.isEmpty() && s.peek() != '(') {
            postfix += s.pop();
         }
         s.pop(); // Remove the '(' from stack
      }
      else {
         while (!s.isEmpty() && getPrecedence(s.peek()) >= getPrecedence(ch)) {
            postfix += s.pop();
         }
         s.push(ch);
      }
   }

   while (!s.isEmpty()) {
      postfix += s.pop();
   }

   return postfix;
}

string infixToPrefix(string expression) {
   reverse(expression.begin(), expression.end());

   for (int i = 0; i < expression.length(); i++) {
      if (expression[i] == '(') expression[i] = ')';
      else if (expression[i] == ')') expression[i] = '(';
   }
```

```
    string prefix = infixToPostfix(expression);

    reverse(prefix.begin(), prefix.end());

    return prefix;
}

int main() {
    string infix = "A+B*(C^D-E)";

    cout << "Infix:   " << infix << endl;
    cout << "Postfix: " << infixToPostfix(infix) << endl;
    cout << "Prefix:  " << infixToPrefix(infix) << endl;

    return 0;
}
```